

Java Grundlagen für Anfänger – Syntax, Semantik & typische Fehler erklärt

Grundlagen der Java-Sprache: Grammatik, Syntax und Semantik

Kurz vorab: Einstiegshürde für Anfänger

Java ist eine strenge, strukturierte Sprache. Bevor man losprogrammieren kann, gibt es ein paar Regeln und Strukturen, die man kennen muss: Klassen, main()-Methode, Syntax Das kann anfangs wie Bürokratie wirken – es müssen „ein paar Hürden überwunden werden“, bevor man Ergebnisse sieht.

Warum das gut ist:

- Die Strenge hilft, von Anfang an sauberen, gut strukturierten Quellcode zu schreiben.
- Fehler fallen früh auf und die Programme sind leichter verständlich und wartbar.

Tipp

Nicht abschrecken lassen – einmal verstanden, ist Java logisch und konsistent. Andere Sprachen erlauben einen schnelleren Start, aber oft entstehen dadurch versteckte Fehler oder unstrukturierter Code.

Nun wollen wir uns ein wenig mit dem Regelwerk von Java auseinandersetzen. Es soll um:

- **Grammatik**
- **Syntax**
- **Semantik**
- **Token**
- **Whitespace**
- **Separatoren**

gehen.

Grammatik in Programmiersprachen

Wenn wir über Grammatik in Programmiersprachen sprechen, meinen wir die Bausteine, die uns die Sprache bereitstellt – also alles, womit wir später Logik umsetzen können.

Beispiele für Bausteine in Java:

- **Schlüsselwörter:** if, else, for, while, int, String
- **Operatoren:** +, -, *, /, %
- **Literale:** Zahlen (42), Text ("Hallo"), Wahrheitswerte (true, false)
- **Separatoren:** ;, {, }, (,)

Merke

- Die Grammatik sagt nur, welche Bausteine existieren.
- Sie sagt noch nicht, wie man sie korrekt kombiniert – das macht erst die Syntax.
- Man kann sich die Grammatik vorstellen wie ein LEGO-Set: Man sieht, welche Steine vorhanden sind, aber weiß nicht, wie sie zusammenpassen, um ein funktionierendes Modell zu bauen.

Grammatik = die Mittel, die die Programmiersprache zur Verfügung stellt, um die gewünschte Logik später umsetzen zu können.

Syntax in der Programmierung

In der Programmierung bezeichnet Syntax die Regeln, die festlegen, wie Quellcode korrekt aufgebaut sein muss.

Wichtigste Punkte:

- Sie bestimmt, wie Wörter, Symbole und Anweisungen angeordnet werden müssen.
- Sie sorgt dafür, dass Programme in derselben Sprache konsistent sind.
- Verstöße gegen die Syntax führen zu Syntaxfehlern – der Compiler oder Interpreter kann den Code dann nicht verstehen.
- Sie legt fest, wann und wie Sprachelemente verwendet werden dürfen.

Man kann sich die Programmiersprache wie eine Kommunikationsschnittstelle zwischen Mensch und Computer vorstellen:

- Menschen verstehen auch falsch formulierte Sätze oft mit etwas Nachdenken.
- Ein Computer kann nur syntaktisch korrekten Code verarbeiten.

Merke

Um ein funktionierendes Programm zu schreiben, muss die Syntax korrekt eingehalten werden.

Semantik in Programmiersprachen

Die Semantik beschreibt die Bedeutung des Quellcodes – also was der Code tatsächlich tut, wenn er ausgeführt wird.

Unterschied zur Syntax

- Syntax prüft, ob der Code korrekt aufgebaut ist (Klammern, Semikolons, Reihenfolge der Schlüsselwörter).
- Semantik wird nicht vom Compiler geprüft – dieser weiß nicht, ob der Code logisch sinnvoll ist.

Beispiele

- Syntax korrekt, Semantik falsch:
`int x = 10;
int y = 0;
int z = x / y; // Syntax okay, semantisch problematisch (Division durch 0)`
- Syntax korrekt, Semantik logisch:
`int x = 10;
int y = 2;
int z = x / y; // alles korrekt, Quellcode tut das, was man erwartet`

Wichtig

- Der Compiler/Interpreter sorgt nur dafür, dass die Grammatik und Syntax stimmen.
- Die Semantik – also die Logik und Sinnhaftigkeit des Programms – liegt komplett beim Programmierer.
- Man kann also beliebigen Quellcode schreiben, solange er grammatikalisch und syntaktisch korrekt ist, auch wenn die Logik keinen Sinn ergibt.

Merke

Semantik = Was der Code tatsächlich bedeutet / tut

Zusatz für Semantik

Manche Fehler, wie **Division durch 0** oder **NullPointerExceptions**, werden zwar erkannt – aber erst zur Laufzeit und nur, weil Java diese Fälle explizit verbietet. Die eigentliche Semantik – also ob der Code das Richtige tut – bleibt immer Verantwortung des Programmierers.

Tokens in der Programmierung

Tokens sind die kleinstmöglichen bedeutungsvollen Einheiten eines Quellcodes. Sie sind die Bausteine, aus denen ein Programm besteht.

Zu den Tokens zählen unter anderem:

- **Schlüsselwörter** (z.B. `if`, `class`, `while`)
- **Bezeichner** (z.B. Variablennamen)
- **Operatoren** (z.B. `+`, `-`, `=`)
- **Literale und Konstanten** (z.B. `42`, `"Hallo"`, `true`)
- **Trenn- und Sonderzeichen** (z.B. `;`, `{`, `}`)

Welche Zeichenfolgen Token bilden dürfen, wird durch die Grammatik der Programmiersprache festgelegt. Die Grammatik definiert also, was als gültiges Token erkannt wird.

Beispiel: Bezeichner

Für einen Bezeichner könnte eine Regel lauten: Lies Zeichen, solange auf einen Buchstaben nur Buchstaben oder Ziffern folgen. So erkennt der Compiler, wo ein Token beginnt und wo es endet.

Warum Tokens wichtig sind

Der Compiler zerlegt den Quellcode zunächst in einzelne Tokens, um ihn überhaupt analysieren zu können. Erst auf Basis dieser Tokens können Syntax und weitere Regeln geprüft und der Code schließlich in ausführbaren Programmcode übersetzt werden.

Formen von Tokens

Ein Token kann dabei ganz unterschiedlich aussehen:

- **ein ganzes Wort** (z.B. Programm)
- **ein Teil eines Wortes** (z.B. "Pro" und "gramm" in einem zusammengesetzten Namen)
- **ein einzelnes Symbol / Zeichen** (z.B. "=", "+", "?")

Merke

Tokens sind die kleinsten Einheiten, die der Compiler "lesen und verstehen" kann.

Whitespace

Als Whitespace bezeichnet man unsichtbare Zeichen wie:

- **Leerzeichen**
- **Zeilenumbrüche**
- **Tabulatoren**

Whitespace dient dazu, Code zu strukturieren und für Menschen besser lesbar zu machen. Zusätzlich hilft er dem Compiler dabei, Tokens voneinander zu trennen.

Abgesehen davon hat Whitespace keine eigene Bedeutung für den Programmablauf. Der Code funktioniert unabhängig davon, wie viel oder wie wenig Whitespace verwendet wird.

Beispiel

Variante Nr. 1

```
int x=10;
```

Variante Nr. 2

```
int x = 10;
```

Variante Nr. 3

```
int
x
=
10
;
```

Alle drei Varianten sind für den Compiler **gleichwertig**.

Wichtig

- Whitespace beeinflusst nicht die Logik des Programms
- Er verbessert ausschließlich Lesbarkeit und Struktur
- Er kann beliebig oft zwischen Tokens eingesetzt werden

Merke

Whitespace ist für Menschen wichtig – Separatoren sind für die Sprache selbst notwendig.

Separatoren (Trennzeichen / Delimiter)

Separatoren sind fest definierte Zeichen, die im Quellcode dazu dienen, Strukturen zu bilden und voneinander abzugrenzen. Sie helfen sowohl dem Compiler als auch dem Programmierer dabei, Code korrekt zu interpretieren und besser zu lesen.

Im Gegensatz zu Whitespace haben Separatoren eine **konkrete Bedeutung** für die Programmiersprache.

Beispiel

In einer Liste: (apfel, banane, birne)

trennen die **Kommas** die einzelnen **Elemente voneinander** und machen klar, wo ein Element endet und das nächste beginnt.

Aufgaben von Separatoren

Separatoren werden verwendet, um:

- Codeblöcke zu strukturieren
- Anweisungen zu beenden
- Parameter, Argumente oder Elemente zu trennen
- Zusammengehörige Codeabschnitte zu kennzeichnen

Sie sind damit ein zentraler Bestandteil der Syntax.

Separatoren in Java

Zu den wichtigsten Separatoren in Java gehören:

- (und) -> Gruppierung von Ausdrücken, Methodenparameter
- { und } -> Codeblöcke (z. B. Klassen, Methoden, Schleifen)
- [und] -> Arrays
- ; -> Abschluss einer Anweisung
- , -> Trennung von Parametern oder Elementen
- . -> Zugriff auf Mitglieder (z. B. objekt.methode())
- ... -> Varargs
- @ -> Annotationen
- :: -> Methodenreferenzen

Merke

Separatoren trennen und strukturieren Quellcode – ohne sie könnte der Compiler den Programmablauf nicht eindeutig erkennen.

Analyse unseres ersten Java-Programms

Schauen wir uns jetzt unser erstes Java-Programm noch einmal an – nicht um es vollständig zu verstehen, sondern um zu erkennen, welche sprachlichen Regeln und Bausteine Java hier verwendet.

```
public class ErstesProgramm {
```

In dieser Zeile sehen wir mehrere Bausteine der Java-Grammatik: `public` und `class` sind Schlüsselwörter in Java. `ErstesProgramm` ist ein Bezeichner, also ein selbst gewählter Name. Die geschweifte Klammer `{` ist ein Separator und markiert den Beginn eines Codeblocks. Der Zeilenumbruch danach ist `Whitespace` und dient nur der Lesbarkeit.

```
public static void main(String[] args) {
```

Auch hier verwenden wir wieder mehrere Bausteine der Sprache: `public`, `static` und `void` (Typangabe des Rückgabewertes) sind fest definierte Schlüsselwörter. `main` ist ein Bezeichner/Methodenname, der den Namen der Methode und gleichzeitig den Einstiegspunkt des Programms setzt.

Die runden Klammern `(` und `)` sind Separatoren für die Methodendeklaration. `String` ist ein Datentyp, `args` ein Bezeichner und die eckigen Klammern `[ ]` kennzeichnen ein Array. Die öffnende geschweifte Klammer `{` startet den nächsten Codeblock.

```
System.out.println("Mein erstes Java-Programm! Änderung");
```

Diese Zeile ist ein Methodenaufruf. Die Punkte `.` sind Separatoren, mit denen wir auf Bestandteile zugreifen. `println` ist der Name der Methode.

Der Text in doppelten Anführungszeichen ist ein `String`. Die runde Klammer `)` beendet den Methodenaufruf und das Semikolon `;` ist ein Separator, der Anweisungen beendet.

```
}  
}
```

Die beiden schließenden geschweiften Klammern `}` beenden die zuvor geöffneten Codeblöcke – zuerst die `main`-Methode, dann die Klasse. Auch hier sorgt `Whitespace` nur für Struktur und Übersicht.

Hieran lässt sich folgendes erkennen:

Java besteht aus festen Bausteinen, klaren Regeln und eindeutigen Trennzeichen. Der Quellcode wird nicht auf Sinn oder Logik geprüft – der Compiler analysiert ausschließlich die formalen Regeln der Sprache.